

1. ¿Qué has aprendido en este proyecto?

A través de este proyecto, he adquirido un conocimiento sobre los protocolos de consistencia débil, específicamente el funcionamiento de Time Stamped Anti-Entropy (TSAE). He aprendido cómo se logra la entrega eventual y fiable de mensajes mediante el intercambio periódico de estados entre réplicas (sesiones de anti-entropía). En el plano técnico, he aprendido a traducir estructuras teóricas (como los vectores de resumen y los registros de operaciones) a una implementación real en Java 17, comprendiendo la importancia crítica de los timestamps para ordenar eventos y evitar "huecos" en la información recibida por cada servidor.

2. ¿Qué parte ha sido la más desafiante y cómo la superaste?

La parte más compleja ha sido garantizar la seguridad frente a hilos (thread-safety) en las estructuras de datos fundamentales. Dado que el hilo de la aplicación y los hilos de red de las sesiones de sincronización acceden simultáneamente al Log y al TimestampVector, existía un riesgo real de corrupción de datos. Lo superé aplicando mecanismos de sincronización rigurosos. Marque los métodos como synchronized, garantizando que:

- Seguridad frente a hilos: Solo un hilo pueda ejecutar cualquier método sincronizado sobre la misma instancia de la clase (Log o TimestampVector) a la vez, lo cual es fundamental ya que las sesiones de anti-entropía y la aplicación pueden interactuar simultáneamente.
- Cumplimiento de restricciones: Te ajustas a la instrucción de ser "extremadamente cauteloso con el acceso concurrente" a las estructuras de datos.

3. ¿Qué concepto no terminaste de entender del todo y te gustaría explorar más a fondo?

Me gustaría profundizar en la transición de un sistema con relojes sincronizados a uno con relojes no sincronizados. Aunque la Fase 1 se centra en la adición de mensajes, el concepto de utilizar una matriz de confirmación (ACK Matrix) con complejidad $O(n^2)$ para permitir la purga de logs sin depender de la sincronización de relojes me interesa. Me gustaría explorar cómo este diseño escala en sistemas con miles de nodos donde el mantenimiento de dicha matriz podría volverse ineficiente.

4. Si tuvieras que repetir el proyecto, ¿qué cambiarías o mejorarías?

Me gustaría modificar el proyecto para hacer uso de mecanismos de sincronización avanzados como ReentrantReadWriteLock que permiten optimizar el rendimiento del protocolo TSAE al separar las operaciones de lectura de las de escritura, manteniendo intacta la arquitectura requerida por las fuentes.

Estas mejoras preservarían la estructura del proyecto e incrementarían la eficiencia. Actualmente, al usar el modificador `synchronized` en los métodos, se bloquea todo el objeto tanto para lecturas como para escrituras. En un sistema distribuido como el descrito en las fuentes, donde las consultas (lecturas) son frecuentes, esto genera cuellos de botella y con esta solución se optimizaría el rendimiento.

Modificaciones en fichero java del proyecto

Fichero Log.java

Función add.

```
public synchronized boolean add(Operation op) {
    // recupero el ID del host
    String hostId = op.getTimestamp().getHostid();
    // recuperar el log del host
    List<Operation> operations = log.get(hostId);

    // log no vacío, recupera última marca de tiempo
    Timestamp lastTimestamp;
    if (!operations.isEmpty())
        lastTimestamp = operations.get(operations.size() - 1).getTimestamp();
    else
        lastTimestamp = null;

    // comparar tiempo actual con la última entrada
    // si son iguales
    if (op.getTimestamp().compare(lastTimestamp) < 0)
        return false;
    // sino actualizo log
    else {
        log.get(hostId).add(op);
        return true;
    }
}
```

Esta función tiene como objetivo insertar nuevas operaciones en el registro de mensajes (log) de forma ordenada y segura, asegurando que se respete la secuencia lógica de los timestamps para cada host emisor. Para esto sigue estos pasos:

- **Identificación del emisor:** Se obtiene el identificador del host (hostId) que originó la operación consultando el Timestamp adjunto a la misma.
- **Acceso al log específico:** Se recupera la lista de operaciones asociadas a ese hostId desde la estructura de datos log.
- **Localizar la última entrada:** Si la lista no está vacía, identifica el timestamp de la última operación registrada para ese host (lastTimestamp). Si está vacía, lo inicializa como nulo.
- **Validar el orden cronológico:** Se compara el timestamp de la nueva operación con el último registrado. Si el nuevo timestamp es "menor" (anterior en el tiempo) que el último, la función devuelve false y rechaza la operación. Esto es perentorio en el protocolo TSAE para garantizar que no existan "saltos/huecos" en la secuencia de mensajes de un emisor; el último timestamp debe representar que se han recibido todos los anteriores.
- **Actualizar del log:** Si el timestamp es válido (igual o posterior al último), la operación se añade al final de la lista del host correspondiente y la función devuelve true.

Se ha modificado la función a `synchronized` por las siguientes razones:

- **Prevención de corrupción de datos:** En la Fase 1 de la práctica, se exige ser estricto con el acceso concurrente, ya que las acciones de diferentes hilos (como una sesión de anti-entropy recibiendo datos de red y la aplicación local añadiendo recetas) pueden entrelazarse.
- **Evitar interferencias:** Sin sincronización, dos hilos podrían intentar modificar la lista de operaciones de un mismo host simultáneamente, lo que podría causar que se perdieran mensajes o que el orden de los timestamps en el log dejara de ser consistente.
- **Integridad del protocolo:** El protocolo TSAE se basa en que el Summary Vector refleje fielmente el estado del log. Si el log se corrompe por accesos concurrentes no controlados, las sesiones de anti-entropy dejarían de ser fiables y los servidores podrían divergir permanentemente.

Función equals.

```
@Override
public boolean equals(Object obj) {
    // Verificar de identidad y nulidad básica
    if (this == obj) return true;
    if (obj == null) return false;
    if (getClass() != obj.getClass()) return false;

    // Casting seguro a la clase Log
    Log other = (Log) obj;

    // Comparar el mapa log de la instancia actual con el de la other
    if (this.log == null) {
        return other.log == null;
    } else return this.log.equals(other.log);
}
```

Con esta función implemento el método `equals` para la clase `Log`, siendo necesaria para implementar el mecanismo de registro de todas las operaciones en el protocolo TSAE. Con el fin de determinar si dos objetos de tipo `Log` son idénticos en contenido, fundamental para verificar si dos réplicas han alcanzado la consistencia eventual. A comento el funcionamiento paso a paso:

- **Verificación de identidad (`if (this == obj)`):** Comprueba si ambas referencias apuntan al mismo objeto en la memoria. Si es así, son idénticos y devuelve `true` inmediatamente para ahorrar tiempo de cómputo.
- **Verificación de nulidad (`if (obj == null)`):** Si el objeto comparado es nulo, no puede ser igual al objeto actual (que obviamente no es nulo), por lo que devuelve `false`.
- **Verificación de clase (`if (getClass() != obj.getClass())`):** Asegura que ambos objetos pertenezcan exactamente a la misma clase (`Log`). Esto evita errores al intentar comparar un `Log` con otro tipo de estructura de datos, como un `TimestampVector`.

- Casting a la clase Log: Una vez confirmado que son del mismo tipo, el objeto obj (que entra como un Object genérico) se convierte al tipo Log para poder acceder a sus atributos internos, específicamente al mapa log que contiene las operaciones.
- Comparación del atributo interno (this.log):
 - Si el mapa log de la instancia actual es nulo, la función solo devuelve true si el mapa log del otro objeto también es nulo.
 - Si no es nulo, utiliza el método .equals() de la estructura de datos interna (el ConcurrentHashMap definido en la estructura de la práctica) para realizar una comparación profunda de los mensajes y timestamps almacenados.

La anotación @Override en la función es fundamental por:

- Sobrescritura de la Clase Base: Indica que este método está redefiniendo el método equals(Object obj) que heredan todas las clases de la clase base java.lang.Object.
- Seguridad y Compilación: Ayuda al compilador a verificar que la firma del método es correcta. Si se cometiera un error en el nombre o los parámetros, el compilador daría un error, lo cual es vital para cumplir con las reglas estrictas de implementación que prohíben desviaciones estructurales en las clases modificadas.
- Evaluación en DSLab: La plataforma DSLab utiliza estos métodos para verificar formalmente que los componentes de la Fase 1 (Log y TimestampVector) funcionan correctamente. Un método equals bien implementado permite asegurar que, tras una sesión exitosa de anti-entropía, ambos principales tienen efectivamente el mismo conjunto de mensajes en sus registros.

Fichero TimestampVector.java

Función updateTimestamp.

```
public synchronized void updateTimestamp(Timestamp timestamp) {
    LSimLogger.log(Level.TRACE, "Updating the TimestampVectorInserting with the
timestamp: "+timestamp);
    //Actualizar tiempo
    timestampVector.put(timestamp.getHostid(), timestamp);
}
```

El comando `timestampVector.put(timestamp.getHostid(), timestamp)` dentro de la función `updateTimestamp` tiene como objetivo fundamental actualizar el estado de conocimiento del servidor local respecto a las operaciones recibidas de otros miembros del grupo.

A continuación se explica el porqué de esta operación detalladamente:

Identificación del Emisor: El método `timestamp.getHostid()` recupera la identidad única del principal (host) que originó la operación. El vector utiliza este ID como clave para asociarle su marca de tiempo más reciente.

Mantenimiento del Summary Vector: En el protocolo TSAE, el objeto `timestampVector` actúa como el Summary Vector. Esta estructura es crítica porque registra qué actualizaciones ha observado la réplica actual de cada uno de sus compañeros.

Garantía de "Sin Huecos": Según las especificaciones del protocolo, registrar un timestamp `t` para un host B en el vector del host A significa que A ha recibido todos los mensajes de actualización enviados por B hasta el tiempo `t`. El uso de `.put` permite que el vector refleje este "corte" en la comunicación, asegurando que no existan huecos en la secuencia de mensajes procesados.

Eficiencia en la Sincronización: Durante las sesiones de anti-entropy, este vector se intercambia entre servidores para determinar rápidamente qué mensajes le faltan a un compañero. Al mantener el vector actualizado con `.put`, el servidor puede informar con precisión qué es lo "nuevo" que posee para compartir.

Control de Concurrencia: El hecho de que la función sea `synchronized` asegura que la actualización del vector mediante el comando `.put` sea atómica y segura frente a hilos. Esto evita que, si varias sesiones de anti-entropy o procesos locales intentan actualizar el conocimiento del servidor al mismo tiempo, se produzca una corrupción de datos o una inconsistencia en el estado del protocolo.

Función equals.

```
@Override
public boolean equals (Object obj) {
    // Verificar de identidad y nulidad básica
    if (this == obj) return true;
    if (obj == null) return false;
    if (getClass() != obj.getClass()) return false;

    // Casting seguro a la clase Log
    TimestampVector other = (TimestampVector) obj;

    // Comparar el mapa log de la instancia actual con el de la other
    if (this.timestampVector == null) {
        return other.timestampVector == null;
    } else return this.timestampVector.equals (other.timestampVector);
}
```

Esta función implementa el método equals para la clase TimestampVector, estructura de datos esencial en el protocolo TSAE que actúa como el Vector de Resumen (Summary Vector). Su propósito es determinar si dos vectores de marcas de tiempo son idénticos, lo que permite verificar si dos réplicas han alcanzado el mismo estado de conocimiento o "corte" en la comunicación. Estos son los pasos que sigue la función:

- Verificación de Identidad (this == obj): Comprueba si ambos objetos son la misma instancia en la memoria [Propia]. Si apuntan a la misma dirección, devuelve true inmediatamente por eficiencia.
- Verificación de Nulidad (obj == null): Si el objeto comparado es nulo, no puede ser igual a la instancia actual, por lo que devuelve false [Propia].
- Verificación de Clase (getClass() != obj.getClass()): Asegura que el objeto recibido sea estrictamente de la clase TimestampVector [Propia]. Esto evita comparar erróneamente un vector con otras estructuras del protocolo, como un Log o una TimestampMatrix.
- Casting Seguro: Convierte el objeto genérico obj al tipo específico TimestampVector (llamado other) para poder acceder a su atributo interno timestampVector [Propia].
- Comparación de Contenido:
 - Si el atributo interno this.timestampVector es nulo, la función solo devuelve true si el del otro objeto también lo es.
 - Si no es nulo, utiliza el método .equals() de la colección interna (que almacena los pares de principalId y timestamp) para realizar una comparación profunda de todas las marcas de tiempo registradas.

La anotación @Override indica que este método está sobrescribiendo el método equals heredado de la clase base java.lang.Object. Con ello conseguimos:

- Seguridad en el Desarrollo: Garantiza que realmente estás redefiniendo el método de la clase base [Propia]. Si la firma del método fuera incorrecta (por ejemplo, si los parámetros no

coincidieran), el compilador mostraría un error, ayudando a cumplir con las reglas estrictas de implementación que prohíben desviaciones estructurales.

- **Consistencia del Protocolo:** En TSAE, la igualdad entre vectores de resumen es fundamental para medir la consistencia eventual. Sin esta sobrescritura, el sistema usaría la comparación por defecto de Java (que solo mira la dirección de memoria), lo que impediría detectar que dos réplicas diferentes han recibido el mismo conjunto de mensajes.
- **Evaluación en DSLab:** La plataforma de evaluación formal requiere que estas estructuras de datos se comporten de acuerdo con las especificaciones del protocolo para validar correctamente el intercambio de estados en las sesiones de anti-entropía.